RAG 원리학습 — NotebookLM 의 블랙박스를 열어보기

강사 시연용 · rag_principle_demo_colab.ipynb 안내서 · 10 주차 연계

왜 이 실습이 필요한가

이 과정에서 학생들은 지금까지 NotebookLM 을 '자료 올리면 알아서 출처와 함께 답해주는 도구'로만 써왔습니다. 이건 활용법으로는 충분하지만, '왜 믿을 수 있는지 / 왜 가끔 틀리는지'에 대한 원리 이해는 남습니다.

이 노트북은 NotebookLM 안에서 실제로 벌어지는 일을 LangChain 코드로 직접 열어봅니다. 학생이 코딩하는 실습이 아니라, 강사가 실행하며 원리를 설명하는 시연용입니다 — 이 과정의 '노코드 활용법 중심' 원칙은 그대로 유지됩니다.

전체 흐름 (RAG 5 단계를 코드로)

단계	코드가 하는 일	학생에게 설명할 포인트
① 지식베이스	검증된 정답 데이터를 문장 단위 '문서'로 준비	이게 NotebookLM 에 올리는 '소스'와 같은 역할
② 청크 분할	문서를 작은 조각으로 나눔	AI 는 문서를 통째로 안 읽고 조각내서 다룸
③ 임베딩·저장	각 문장을 숫자 벡터로 바꿔 저장(FAISS)	'의미가 비슷한 문장 찾기'가 가능해지는 순간
④ 검색(Retrieval)	질문과 가장 가까운 청크를 찾아냄	이 결과가 바로 NotebookLM 의 [1][2] 출처
⑤ 생성(선택)	찾은 청크를 근거로 LLM 이 답변 문장을 만듦	여기서부터가 우리가 보는 '자연스러운 답변'

핵심 시연 — 좋은 질문 vs 근거 없는 질문

이 노트북에서 가장 중요한 순간입니다. 정답 데이터에 실제로 있는 사실과, 지난 실습에서 AI 가 지어냈던 '관계자 코멘트'를 각각 검색해서 유사도를 비교합니다.

질문	결과
9 월 25 일 광주전남 최고기온은?	유사도 거리 낮음 — 정답 데이터와 정확히 일치하는 문서를 찾아냄
기상청 관계자가 뭐라고 코멘트했어?	유사도 거리 높음(=관련성 낮음) — 정답 데이터 어디에도 없는 내용이기 때문

> *이 대목에서 강조할 결론: 제대로 설계된 RAG 라면 두 번째 질문에서 '근거를 찾지 못했다'는 신호를 냈어야 합니다. 그런데 도메인별 스크립트봇은 이 신호를 무시하고 그럴듯한 코멘트를 지어냈습니다 — 지난 팩트체크 실습의 할루시네이션이 '왜' 일어났는지를 코드 수준에서 증명하는 순간입니다.*

실행 중 나올 수 있는 화면 — 당황하지 않아도 되는 것들

1 단계(설치) — 의존성 경고

```
ERROR: pip's dependency resolver does not currently take into account all the packages
that are installed. ... google-colab 1.0.0 requires requests==2.32.4, but you have
requests 2.34.2 which is incompatible.
```

> *이건 실제 오류가 아니라 사전 경고입니다. 설치 자체는 정상 완료되며("설치 완료" 출력됨), 이 데모 코드(LangChain·FAISS)와는 무관합니다. 그대로 다음 셀로 넘어가면 됩니다. 혹시 이후 코랩의 다른 기능(파일 업로드 등)에서 이상 동작이 보일 때만 아래로 되돌리면 됩니다: !pip install -q requests==2.32.4*

4 단계(임베딩 모델 다운로드)는 처음 실행 시 1~2 분 정도 걸릴 수 있습니다 — 멈춘 게 아니라 모델을 내려받는 중입니다.

6 단계(선택, 실제 생성) — 모델명 오류

```
GoogleModelNotFoundError: Error calling model 'gemini-2.5-flash' (NOT_FOUND): 404
NOT_FOUND. This model models/gemini-2.5-flash is no longer available to new users.
```

> *Gemini 모델명은 구글이 수시로 교체합니다. 에러 메시지에 항상 '대신 이 모델을 쓰라'는 안내가 함께 나오니(예: models/gemini-3.6-flash), 코드의 model= 부분을 그 이름으로 바꾸면 됩니다. 노트북은 이미 최신 이름으로 반영해뒀지만, 나중에 또 바뀌면 같은 방식으로 대응하면 됩니다.*

6 단계 — 근거 있는 질문까지 '모른다'로 답하는 경우

원인: 코드에 '거리 점수가 400 보다 작을 때만 근거로 쓴다'는 숫자 기준이 있었는데, 이 숫자는 테스트용 가짜 임베딩 기준으로 정한 것이라 실제 임베딩 모델의 거리 스케일과 맞지 않았습니다. 그 결과 진짜 근거까지 걸러졌습니다.

> *수정: 숫자 기준으로 거르는 대신, 상위 4 개 근거를 항상 그대로 프롬프트에 주고 '근거에 없으면 모른다고 답하라'는 지시만으로 판단하게 바꿨습니다. 노트북은 이미 이 수정이 반영되어 있습니다.*

기대되는 정상 출력 예시

수정된 코드로 6 단계를 정상 실행하면 아래처럼 나와야 합니다.

```
[질문 1 답변]
30 도입니다.

[질문 2 답변]
모른다
```

> *이 결과가 이 노트북 전체의 결론입니다 — 같은 Gemini 모델인데도, 프롬프트에 '근거 없으면 모른다고 답하라'는 지시 하나만 있으면 할루시네이션 없이 정직하게 답합니다. 도메인별 스크립트봇이 관계자 코멘트를 지어낸 건 모델의 한계가 아니라, 이런 지시(그리고 그 지시를 지키게 만드는 절대규칙)가 없었기 때문이라는 걸 코드로 증명하는 셈입니다.*

응용 시연 — 다른 주제로 재현하기 (KOSIS 고령인구비율 데이터)

같은 원리가 기상 데이터가 아닌 전혀 다른 도메인(인구통계)에서도 그대로 재현되는지 검증한 변형 실습입니다. 지식베이스를 기상 예보 데이터 대신 KOSIS 지역별 고령인구비율 데이터(전국 17 개 시도, 2026

년 8 월 기준)로 교체하고, 학생이 실제로 CSV 파일을 업로드해서 지식베이스를 구성하는 과정까지 포함했습니다.

지식베이스 구성

kosis_지역별_고령인구비율_202608.csv(17 개 지역, 전체_고령인구비율 컬럼)를 업로드한 뒤, 지역 하나당 문장 하나("OO 의 전체 고령인구비율은 OO%입니다.")로 변환해 17 개의 Document 로 만듭니다. 청크 분할(chunk_size=100)을 거쳐도 문장이 짧아 17→17 로 변화가 없는데, 이 자체가 '청크 분할은 긴 문서를 쪼갤 때 의미가 있고, 이미 짧은 문장 단위 데이터에는 별 효과가 없다'는 걸 보여주는 부수적인 학습 포인트입니다.

시연 1·2 — 근거 있는 질문 vs 근거 없는 질문 (기존 데모와 동일한 구도)

질문	검색 결과	지시 있음	지시 없음
세종특별자치시의 고령인구비율은 몇 %야?	거리 89.11 — 세종 문서를 정확히 찾음	"12.89%입니다." (정답)	—
경상북도 고령화가 심한 이유가 뭐야?	거리 134.95 이상 — 근거에 없고 엉뚱한 지역(충북·충남)이 검색됨	"모른다" (정직한 거부)	근거 없음을 스스로 밝히면서도 그럴듯한 일반론을 지어내 답변

새로운 발견 — '고지된 할루시네이션'

지시 없이 답한 경우, 모델은 "제공된 참고 자료에는 관련 내용이 없습니다"라고 먼저 밝힌 뒤에도 결국 지어낸 설명(청년층 유출·농어촌 비중·저출산 등)을 이어붙였습니다. 이는 기존 데모의 '관계자 코멘트' 사례보다 한층 더 주의가 필요한 패턴입니다 — 대놓고 지어내는 게 아니라 "근거는 없지만 일반적으로는"이라는 정직한 전제를 앞세우기 때문에, 오히려 더 신뢰가 가 보이는 문장이 만들어집니다. 방송에서 전문가 코멘트를 이런 식으로 보완하면, 시청자는 그 설명이 검증된 자료 기반인지 모델의 일반 지식인지 구분할 수 없습니다.

> *결론은 기존 데모와 동일합니다 — 차이를 만드는 것은 모델의 한계가 아니라 '근거 없으면 모른다고만 답하라'는 절대 지시의 유무입니다.*

시연 3 — 집계형 질문에서 드러나는 RAG 의 구조적 한계 (신규)

"고령화가 가장 높은 지역과 가장 낮은 지역을 알려줘"처럼 전체 데이터를 다 훑어야 답이 나오는 집계·통계형 질문을 던지면 전혀 다른 문제가 드러납니다.

단계	실제 결과
검색(k=3)	전남광주통합특별시(25.04%), 전북특별자치도(27.55%), 충청남도(24.36%) — 실제 최고치(경상북도 28.57%)도 최저치(세종 12.89%)도 포함되지 않음
원인	유사도 검색은 질문 문장과 '의미가 비슷한' 문서를 찾을 뿐, 17 개 지역의 수치를 비교해 최댓값·최솟값을 계산해주지 않음

> *결론: RAG 는 "이 지역의 값이 얼마야?" 같은 개별 사실 확인·설명형 질문에는 강하지만, "가장 높은/낮은/평균은?" 같은 집계·통계형 질문에는 구조적으로 맞지 않습니다. k 를 늘려 전체 문서를 다 넣는 건 사실상 검색을 포기하는 것과 같으므로, 이런 질문은 판다스 등으로 데이터를 직접 계산하는 방식(국가통계포털_KOSIS_지역통계_분석_실습가이드.docx 참고)이 맞는 접근입니다. NotebookLM 이나 사내 스크립트봇에 "1 위/최고/평균"을 묻고 그 답을 그대로 기사에 쓰면 안 되는 이유가 바로 이것입니다.*

응용 시연에서 새로 확인된 트러블슈팅

증상	원인 및 해결
ModuleNotFoundError: No module named 'langchain.docstore'	최신 LangChain 은 Document 클래스가 langchain_core.documents 로 이전됨. from langchain_core.documents import Document 로 수정
CSV 업로드 후 FileNotFoundError, os.listdir(".")도 빈 목록	코랩 좌측 파일 탐색기의 업로드 버튼은 '현재 보고 있는 폴더'에 저장되는데, 이게 실제 작업 폴더(/content)가 아니라 파일시스템 최상위(/)일 수 있음. from google.colab import files; files.upload()로 코드에서 직접 업로드하면 항상 노트북의 실제 작업 폴더에 저장되어 안전함
response.content 가 문자열이 아니라 [{'type': 'text', ...}] 형태로 출력됨	최신 Gemini(3.x) 모델이 응답에 'thought signature'라는 내부 메타데이터를 함께 반환하기 때문. isinstance(content, str)로 분기해서 dict 리스트일 땐 'text' 필드만 이어붙이는 추출 함수 필요
429 RESOURCE_EXHAUSTED (일일 한도 20 회, 무료 티어)	분당 제한과 별개로, 무료 티어는 모델당 하루 20 회라는 낮은 일일 한도가 있음. 짧은 대기 후 재시도로 해결되지 않으며, 강의 중 여러 질문을 연달아 라이브 데모하면 도중에 막힐 수 있음 — 데모 질문 개수를 미리 계획하거나 유료 전환/API 키 예비분을 준비해두는 것이 안전

부록: Gemini API 키 발급 시 흔한 문제

Google AI Studio(aistudio.google.com/apikey)에서 API 키를 처음 발급받을 때 아래 문제를 겪을 수 있습니다.

① "Failed to create project" 오류

- 구글 AI Studio 가 새 프로젝트를 자동으로 만들다가 실패하는, 비교적 흔한 서버측 오류입니다.
- 해결: console.cloud.google.com 에 직접 접속해서 프로젝트를 먼저 만든 뒤, AI Studio 로 돌아와 '가져온 프로젝트 선택'에서 그 프로젝트를 선택합니다.
- 회사·기관 이메일(조직 계정)로 로그인되어 있다면 관리자가 프로젝트 생성 권한을 막아뒀을 수 있습니다 — 개인 지메일 계정으로 시도해보세요.
- 프로젝트 이름에는 한글을 쓸 수 없습니다 (영문자·숫자·하이픈·따옴표·공백·느낌표만 허용).

② "프로젝트 한도에 도달했습니다" 오류

- 예전에 다른 구글 서비스(Apps Script 고급 서비스, Firebase 등)를 쓰다가 자동 생성된 프로젝트들이 쌓여있을 때 발생합니다.
- console.cloud.google.com/cloud-resource-manager 에서 안 쓰는 기존 프로젝트를 삭제해서 여유를 만듭니다.
- ⚠️ 주의: 삭제해도 30 일간 '삭제 대기' 상태로 남아있고, 이 기간에도 한도에 그대로 잡힙니다. 즉시 여유가 필요하면 새로 만들지 말고 이미 남아있는 기존 프로젝트(삭제하지 않은 것)를 그대로 재사용하는 게 가장 빠릅니다.

③ 구글 클라우드 '프로젝트'와 구글 드라이브의 관계

완전히 별개입니다. 드라이브는 파일 저장 공간이고, 클라우드 프로젝트는 API 사용 권한·사용량을 관리하는 단위입니다. 프로젝트를 삭제해도 드라이브 파일에는 전혀 영향이 없습니다.

참고: NotebookLM vs 파이썬 직접 구현 — 언제 무엇을 쓰나

	NotebookLM (노코드)	파이썬 직접 구현 (오늘 실습)
장점	설치·유지보수 불필요, 다양한 형식 지원, 구글이 계속 개선	안전장치를 코드로 세밀하게 강제 가능 (예: 근거 없으면 반드시 모른다고 답하기)
단점	블랙박스라 세부 규칙을 못 넣음 (관계자 코멘트 할루시네이션 사례 있음)	패키지 충돌·모델명 변경 등 직접 대응 필요, API 비용 별도 발생
적합한 용도	실무(실제 기사 작성·팩트체크)	교육(원리 이해) — 왜 NotebookLM 이 이렇게 작동하고, 왜 가끔 틀리는지 이해하기

> *결론: 둘은 경쟁 관계가 아니라 역할이 다릅니다. 실무에는 NotebookLM(또는 도메인별 스크립트봇)을 쓰되, 이 노트북으로 배운 '근거 없으면 모른다고 말해야 한다'는 원칙을 사람이 최종 검증 단계에서 확인하는 것이 최선의 조합입니다.*

수업에서 활용하는 법

1. 10 주차 강의안의 'RAG 개념' 슬라이드(질문입력→관련자료검색→근거와함께답변 다이어그램) 직후에 배치합니다.

2. 학생에게 코드를 따라 치게 하지 말고, 화면을 공유한 채 강사가 셀을 하나씩 실행하며 결과를 같이 봅니다.

3. 5 단계(좋은 질문 vs 나쁜 질문)에서 잠깐 멈추고, 학생들에게 '이 다음에 AI 가 뭐라고 답해야 정직한 것일까'를 먼저 물어본 뒤 결과를 보여주면 효과적입니다.

4. 시간 여유가 있다면 응용 시연(KOSIS 고령인구비율)까지 이어서, 같은 원리가 다른 도메인에서도 재현된다는 것과 '집계형 질문에는 RAG 가 안 맞는다'는 결론까지 보여줍니다.

5. 이후 NotebookLM 실전 실습으로 넘어가서, 방금 코드로 본 원리가 그 도구 안에서 실제로 어떻게 작동하는지 연결합니다.

연계 자료

- 노트북 파일: rag_principle_demo_colab.ipynb
- 강의안: week10_lecture.pptx (RAG 개념 섹션)
- 실습 워크시트: 10 주차_실습워크시트.docx
- 관련 사례: AI 방송대본_팩트체크_실습워크시트.docx / 해답및강사가이드.docx